



DataArt

Go in 90 minutes: from zero to concurrent API

A hands-on workshop for programmers

Mateusz Krupa

Why Go?

- Created at **Google** (2009) by Rob Pike, Ken Thompson, Robert Griesemer
- **Fast compilation** – large projects build in seconds
- **Static binaries** – single file, no runtime dependencies
- **Goroutines** – lightweight concurrency built into the language
- **Rich standard library** – HTTP server, JSON, crypto, testing all included
- Used by: Docker, Kubernetes, Terraform, GitHub CLI

Go vs Others

Feature	Go	Java	Python	C
Garbage collected	✓	✓	✓	×
Static typing	✓	✓	×	✓
Fast compile	✓	×	N/A	×
Simple syntax	✓	×	✓	×
Built-in concurrency	✓ (goroutines)	×	×	×
Single binary output	✓	×	×	✓

GC like Java, syntax from C, simplicity like Python, unique concurrency model

Hello World

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     fmt.Println("Hello, World!")
7 }
```

`package main` – every Go file starts with a package declaration

`import "fmt"` – bring in the formatting/printing package

`func main()` – the entry point; no args, no return value

`fmt.Println` – call an exported function (capital letter = public)

Run it:

```
1 go run main.go
```

Types

```
1  bool
2
3  string
4
5  int  int8  int16  int32  int64
6  uint uint8 uint16 uint32 uint64 uintptr
7
8  byte // alias for uint8
9  rune // alias for int32 – represents a Unicode code point
10
11 float32 float64
```

`bool` – `true` or `false`, zero value is `false`

`string` – immutable UTF-8 text, zero value is `""`

`int` / `uint` families – signed and unsigned integers of various sizes

`byte` = `uint8` ; `rune` = `int32` – one Unicode code point (e.g. `'A'` , `'ž'` , `'🚀'`)

`float32` , `float64` – IEEE 754 floating point

Variables

```
1 // Explicit declaration (var keyword + type)
2 var name string = "Go"
3 var age int = 15
4
5 // Short declaration (type inferred by compiler)
6 active := true
7 count := 42
8
9 // Multiple assignment
10 x, y := 10, 20
```

`var x type = value` – explicit, use when type matters

`x := value` – short form, most common inside functions

Multiple values in one line – useful with function returns

Constants

```
1  const Pi = 3.14159
2  const Greeting = "Hello"
3  const Debug = false
4
5  const (
6      StatusOK      = 200
7      StatusNotFound = 404
8  )
```

- Declared with `const` keyword (not `:=`)
- Can be character, string, boolean, or numeric values
- Evaluated at compile time – cannot be changed at runtime

Arrays & Slices

```
1 // Array – fixed size, rarely used directly
2 var scores [3]int = [3]int{90, 85, 77}
3 fmt.Println(len(scores)) // 3
4
5 // Slice – dynamic, backed by an array
6 langs := []string{"Go", "Rust", "Python"}
7
8 // Append (grows automatically)
9 langs = append(langs, "Zig")
10
11 // Slicing (half-open: includes start, excludes end)
12 first2 := langs[0:2] // ["Go", "Rust"]
```

`[3]int` – array: fixed size, part of the type

`[]string` – slice: dynamic length, what you'll use 99% of the time

`append` – grows the slice, returns a new one

`s[low:high]` – sub-slice, shares underlying memory

Maps

```
1 // Create a map
2 versions := map[string]int{
3     "Go": 1,
4     "Python": 3,
5 }
6
7 // Read a value
8 v := versions["Go"] // 1
9
10 // Write / update
11 versions["Rust"] = 1
12
13 // Check if key exists
14 val, ok := versions["Java"]
15 // ok = false, val = 0 (zero value)
```

`map[K]V{}` – built-in hash map, keys must be comparable

Read with `m[key]` – returns zero value if missing

Write with `m[key] = value`

Comma-ok idiom – `val, ok := m[key]` to check existence

For Loop

```
1 // Classic for loop (only loop in Go!)
2 for i := 0; i < 5; i++ {
3     fmt.Println(i)
4 }
5
6 // While-style (just drop init and post)
7 for n > 0 {
8     n--
9 }
10
11 // Forever (infinite loop)
12 for {
13     // break when done
14 }
```

`for init; condition; post` – the only loop keyword in Go

Drop the init/post → behaves like `while`

Drop everything → infinite loop, use `break` to exit

For Range

```
1 // Range over a slice
2 langs := []string{"Go", "Rust", "Python"}
3 for i, lang := range langs {
4     fmt.Printf("%d: %s\n", i, lang)
5 }
6
7 // Range over a map
8 for key, value := range myMap {
9     fmt.Println(key, value)
10 }
```

- `range` gives you index + value (slice) or key + value (map)
- Use `_` to discard: `for _, v := range items`

If / If-Else

```
1  if x > 0 {  
2      fmt.Println("positive")  
3  }  
4  
5  if x > 0 {  
6      fmt.Println("positive")  
7  } else {  
8      fmt.Println("non-positive")  
9  }  
10  
11 // If with init statement (scoped variable)  
12 if err := doSomething(); err != nil {  
13     fmt.Println("failed:", err)  
14 }
```

No parentheses around condition – braces are required

Standard if/else – no ternary operator in Go

`if init; condition` – variable scoped to the if block

Switch

```
1 // Switch on a value (no break needed – implicit)
2 switch day {
3   case "Monday":
4     fmt.Println("start of week")
5   case "Friday":
6     fmt.Println("almost weekend")
7   default:
8     fmt.Println("regular day")
9 }
10
11 // Switch without a value (cleaner if/else chain)
12 switch {
13   case score ≥ 90:
14     fmt.Println("A")
15   case score ≥ 80:
16     fmt.Println("B")
17   default:
18     fmt.Println("C or below")
19 }
```

- No `break` needed – cases don't fall through by default
- `switch {}` (no value) replaces long if/else chains

Functions

```
1 // Multiple return values
2 func divide(a, b float64) (float64, error) {
3     if b == 0 {
4         return 0, errors.New("division by zero")
5     }
6     return a / b, nil
7 }
8
9 func main() {
10     result, err := divide(10, 3)
11     if err != nil {
12         fmt.Println("Error:", err)
13         return
14     }
15     fmt.Printf("10 / 3 = %.2f\n", result)
16 }
```

Error Handling

```
1 // Functions return (value, error)
2 result, err := divide(10, 0)
3
4 // The classic Go pattern: check err immediately
5 if err != nil {
6     fmt.Println("something went wrong:", err)
7     return
8 }
9 fmt.Println(result)
```

- No exceptions – errors are just values
- Always check `err != nil` right after the call
- `nil` error means success

Structs

```
1 // Exported struct (capital S) – visible outside package
2 type Server struct {
3     Host string // Exported field (capital H)
4     Port int    // Exported field (capital P)
5     active bool // unexported field (lowercase) – private
6 }
7
8 // Unexported struct – only usable within this package
9 type config struct {
10     timeout int
11 }
```

`type Name struct` – defines a composite type with named fields
Lowercase first letter = unexported (private to the package)

Methods

```
1  type Rectangle struct {  
2      Width  float64  
3      Height float64  
4  }  
5  
6  // Value receiver – gets a copy of the struct  
7  func (r Rectangle) Area() float64 {  
8      return r.Width * r.Height  
9  }  
10  
11 // Pointer receiver – can modify the struct  
12 func (r *Rectangle) Scale(factor float64) {  
13     r.Width *= factor  
14     r.Height *= factor  
15 }
```

(r Rectangle) – value receiver, read-only access

(r *Rectangle) – pointer receiver, can mutate fields

Struct Literals & Constructors

```
1 // Named fields (preferred – order doesn't matter)
2 rect := Rectangle{Width: 5, Height: 3}
3
4 // Zero value – all fields get their zero value
5 var empty Rectangle // Width: 0, Height: 0
6
7 // Constructor function (convention: NewXxx)
8 func NewRectangle(w, h float64) *Rectangle {
9     return &Rectangle{
10         Width: w,
11         Height: h,
12     }
13 }
```

Named fields – explicit and safe when struct changes

Zero values – no null, every type has a usable default

`NewXxx` – constructor convention, returns a pointer



Project 1: Calculator

Build a CLI calculator package with tests

Your tasks:

1. Implement `Add`, `Sub`, `Mul`, `Div` in `calculator.go`
2. Handle division by zero with an error return
3. Run `go test ./...` until all tests pass
4. Try the CLI: `go run . add 3 5`

Directory: `01-calculator/starter/`



Click to start (25 min)

defer

```
1 func checkURL(url string) CheckResult {  
2     resp, err := http.Get(url)  
3     if err != nil { /* ... */ }  
4     defer resp.Body.Close() // runs when checkURL returns, even on error  
5  
6     // ... read resp  
7 }
```

- `defer` schedules a call to run when the surrounding function returns
- Always runs – even if the function returns early
- Canonical use: close files, unlock mutexes, release resources

Goroutines

```
1  package main
2
3  import (
4      "fmt"
5      "time"
6  )
7
8  func sayHello(name string) {
9      fmt.Printf("Hello, %s!\n", name)
10 }
11
12 func main() {
13     // Launch a goroutine - runs concurrently
14     go sayHello("World")
15     go sayHello("Go")
16     go sayHello("Concurrency")
17
18     // Wait for goroutines to finish (crude approach)
19     time.Sleep(100 * time.Millisecond)
20 }
```

`go fn()` launches a lightweight concurrent function. Thousands cost almost nothing.

Channels

```
1  package main
2
3  import "fmt"
4
5  func producer(ch chan← string) {
6      ch ← "hello from goroutine"
7  }
8
9  func main() {
10     // Create a channel
11     ch := make(chan string)
12
13     // Send on a goroutine
14     go producer(ch)
15
16     // Receive (blocks until value arrives)
17     msg := ←ch
18     fmt.Println(msg)
19 }
```

Channels are typed conduits for communication between goroutines.

WaitGroup Pattern

```
1 func worker(id int, wg *sync.WaitGroup) {
2     defer wg.Done()
3     fmt.Printf("Worker %d done\n", id)
4 }
5
6 func main() {
7     var wg sync.WaitGroup
8
9     for i := 1; i ≤ 5; i++ {
10         wg.Add(1)
11         go worker(i, &wg)
12     }
13
14     wg.Wait() // Block until all workers call Done()
15     fmt.Println("All workers finished")
16 }
```

- `Add(1)` before launching each goroutine
- `defer wg.Done()` inside the goroutine
- `wg.Wait()` blocks until counter reaches zero

Mutex – Protecting Shared State

```
1  type Store struct {  
2      mu      sync.Mutex  
3      history []string  
4  }  
5  
6  func (s *Store) Add(entry string) {  
7      s.mu.Lock()  
8      defer s.mu.Unlock()  
9  
10     s.history = append(s.history, entry)  
11 }  
12  
13 func (s *Store) GetAll() []string {  
14     s.mu.Lock()  
15     defer s.mu.Unlock()  
16  
17     result := make([]string, len(s.history))  
18     copy(result, s.history)  
19     return result  
20 }
```

- `sync.Mutex` – only one goroutine can hold the lock at a time
- `defer s.mu.Unlock()` – guarantees unlock even on early return

JSON + Struct Tags

```
1 // Struct tags control JSON field names
2 type CheckRequest struct {
3     URLs []string `json:"urls"`
4 }
5
6 // Decode: request body → struct
7 var req CheckRequest
8 json.NewDecoder(r.Body).Decode(&req)
9
10 // Encode: struct → response
11 w.Header().Set("Content-Type", "application/json")
12 json.NewEncoder(w).Encode(req)
```

- ``json:"name"`` – maps struct field to JSON key
- `NewDecoder` reads from any `io.Reader` (request body, file, ...)
- `NewEncoder` writes to any `io.Writer` (response, stdout, ...)

HTTP Handlers

```
1  mux := http.NewServeMux()
2
3  mux.HandleFunc("POST /check", func(w http.ResponseWriter, r *http.Request) {
4      // r.Body - incoming request body (io.Reader)
5      // w        - outgoing response (io.Writer + headers)
6      w.WriteHeader(http.StatusOK) // 200, default if not set
7  })
8
9  http.ListenAndServe(":8080", mux)
```

- "POST /check" – method + path routing (Go 1.22+)
- `w` then `r` – always in this order
- Handler stubs already wired in `main.go` – you fill in the body



Project 2: Link Checker API

Build an HTTP API that checks URLs concurrently

Your tasks:

1. **Phase 1** – `Store.Add` + `Store.GetHistory` with mutex
2. **Phase 2** – `POST /check` sequential (JSON in → checkURL → JSON out)
3. **Phase 3** – make it concurrent: goroutines + channel + WaitGroup
4. **Phase 4** – `GET /history`

Directory: `02-linkchecker/starter/`



Click to start (40 min)

What You Built Today

-  Go syntax, types, and error handling
-  Table-driven tests with `go test`
-  Goroutines, channels, and WaitGroup
-  A concurrent HTTP API from scratch

From zero Go to a concurrent API in 90 minutes 🎉

Resources

Continue your Go journey:

-  **A Tour of Go** – go.dev/tour
-  **Go by Example** – gobyexample.com
-  **Effective Go** – go.dev/doc/effective_go
-  **Go Playground** – go.dev/play
-  **Standard Library Docs** – pkg.go.dev/std

Solutions are in the repo: `01-calculator/solution/` and `02-linkchecker/solution/`

Thank you! 🎉

Questions?